

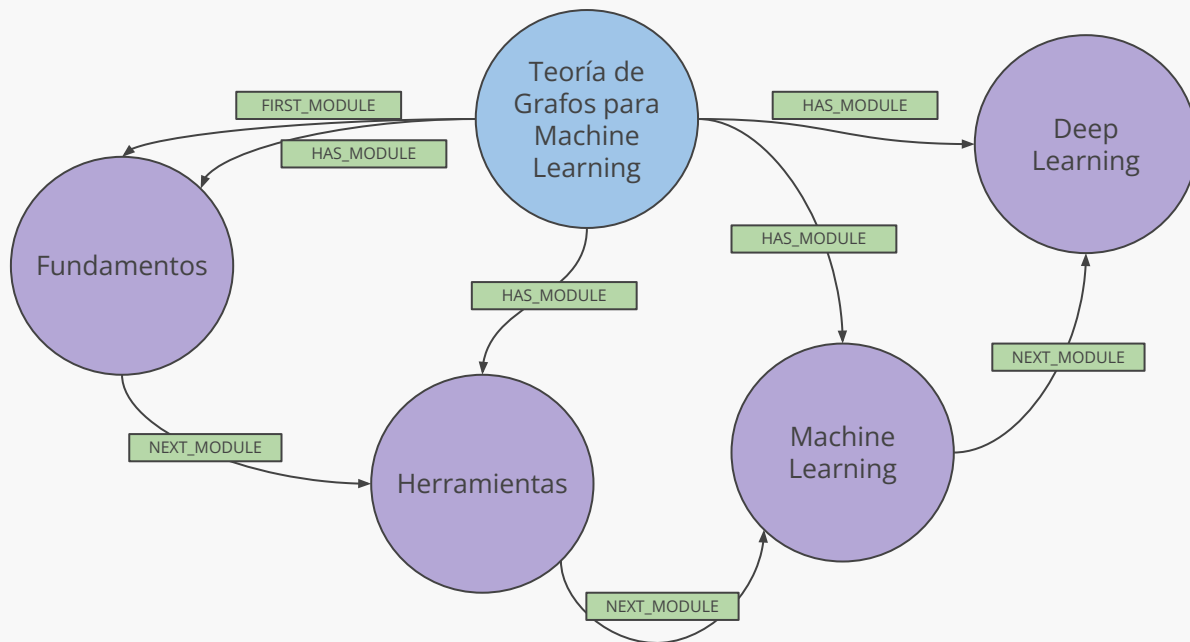
Clase 7.1

Entrenando GNNs

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Contenidos del curso

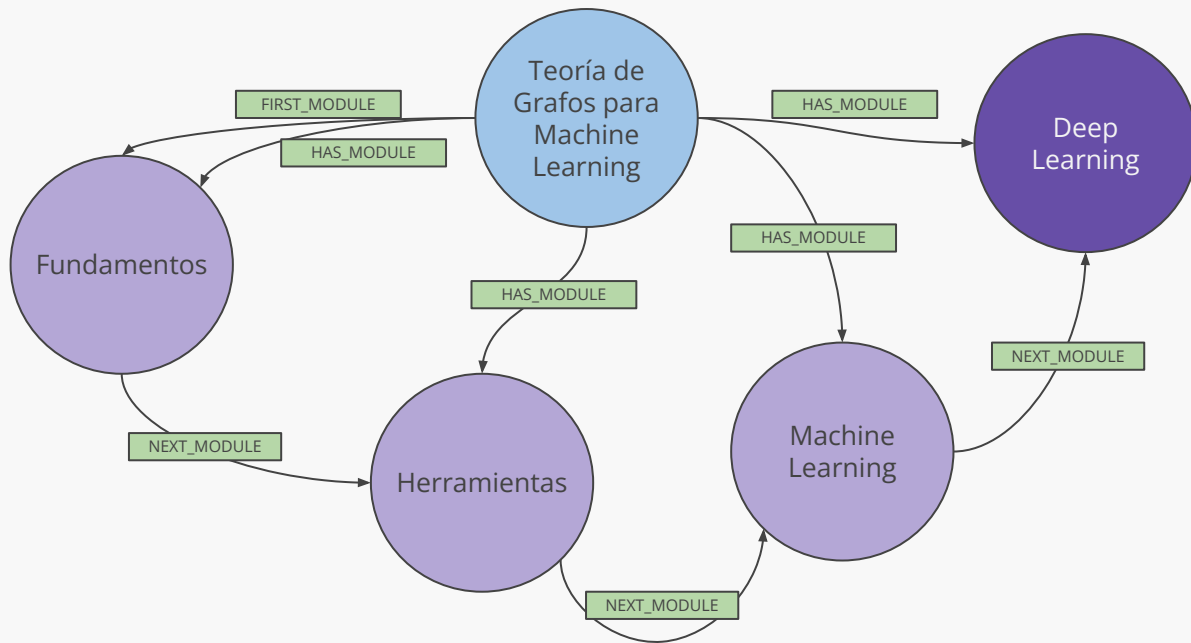


Contenidos del curso

4. Deep Learning

Explorar cómo se extiende el concepto de ML para implementar modelos especializados en grafos (GNNs)...

... de manera de aprovechar lo más posible la estructura y naturaleza de un grafo



Agenda

✨ *Data augmentation*

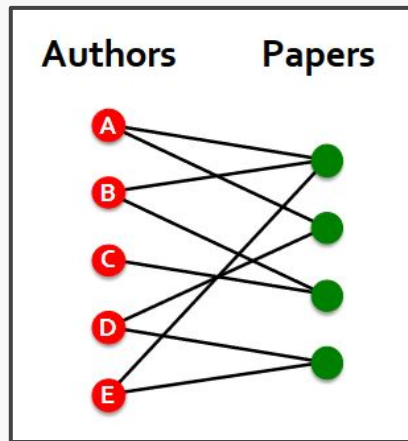
Data splits

Predicción con GNNs

Arcos virtuales

En grafos *sparse*, podemos aumentar el conjunto de entrenamiento y los grafos de cómputo agregando arcos:

- **Por ejemplo**, conectamos vecinos a distancia 2 por medio de arcos virtuales (acercándose artificialmente, saltándose al vecino en común)
- **Es decir**, en lugar de usar la matriz de adyacencia A , estaríamos usando una nueva matriz de adyacencia $A+A^2$
- **Ejemplos de uso:**
 - Grafo bipartito de autores y papers
 - Arcos virtuales (2-hop) representan colaboración autor-autor

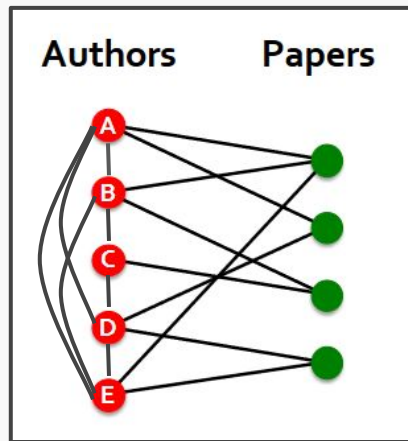


El grafo original es bipartito

Arcos virtuales

En grafos *sparse*, podemos aumentar el conjunto de entrenamiento y los grafos de cómputo agregando arcos:

- **Por ejemplo**, conectamos vecinos a distancia 2 por medio de arcos virtuales (acercándose artificialmente, saltándose al vecino en común)
- **Es decir**, en lugar de usar la matriz de adyacencia A , estaríamos usando una nueva matriz de adyacencia $A+A^2$
- **Ejemplos de uso:**
 - Grafo bipartito de autores y papers
 - Arcos virtuales (2-hop) representan colaboración autor-autor

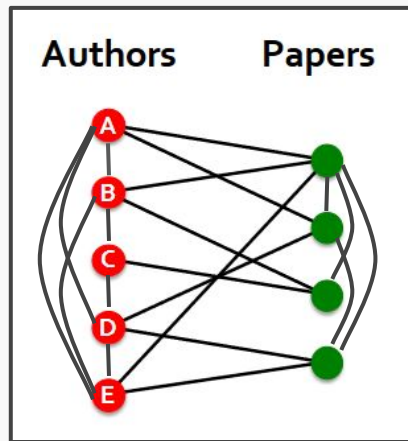


Agregamos arcos virtuales entre autores

Arcos virtuales

En grafos *sparse*, podemos aumentar el conjunto de entrenamiento y los grafos de cómputo agregando arcos:

- **Por ejemplo**, conectamos vecinos a distancia 2 por medio de arcos virtuales (acercándose artificialmente, saltándose al vecino en común)
- **Es decir**, en lugar de usar la matriz de adyacencia A , estaríamos usando una nueva matriz de adyacencia $A+A^2$
- **Ejemplos de uso:**
 - Grafo bipartito de autores y papers
 - Arcos virtuales (2-hop) representan colaboración autor-autor

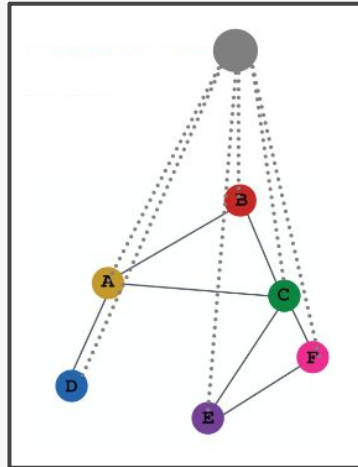
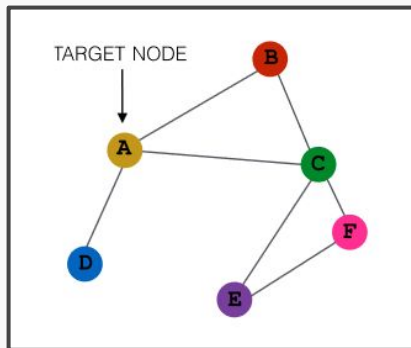


Podemos agregar arcos en los papers

Nodos virtuales

En grafos *sparse*, podemos aumentar el conjunto de entrenamiento y los grafos de cómputo agregando nodos:

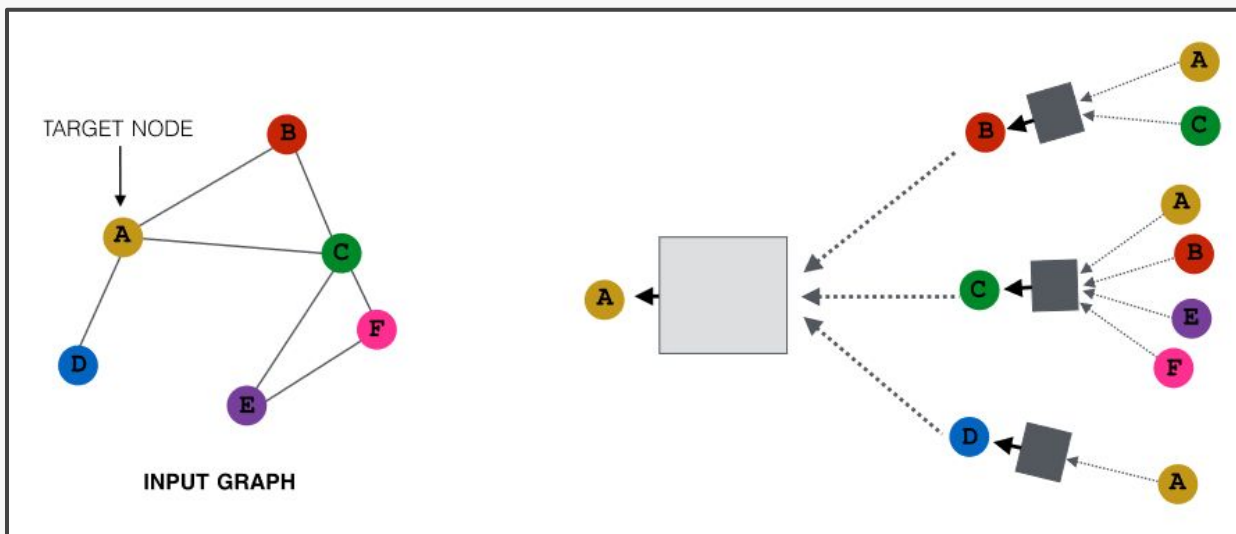
- Un nodo virtual que conecte a todos los nodos en el grafo
- Reduce distancia mínima entre pares de nodos a distancia 2
- Esto **mejora el paso de mensajes**



Muestreo en vecindarios

Usamos una selección aleatoria de vecinos (para pasar sus mensajes) reduciendo el costo computacional (además, funciona bien en la práctica)

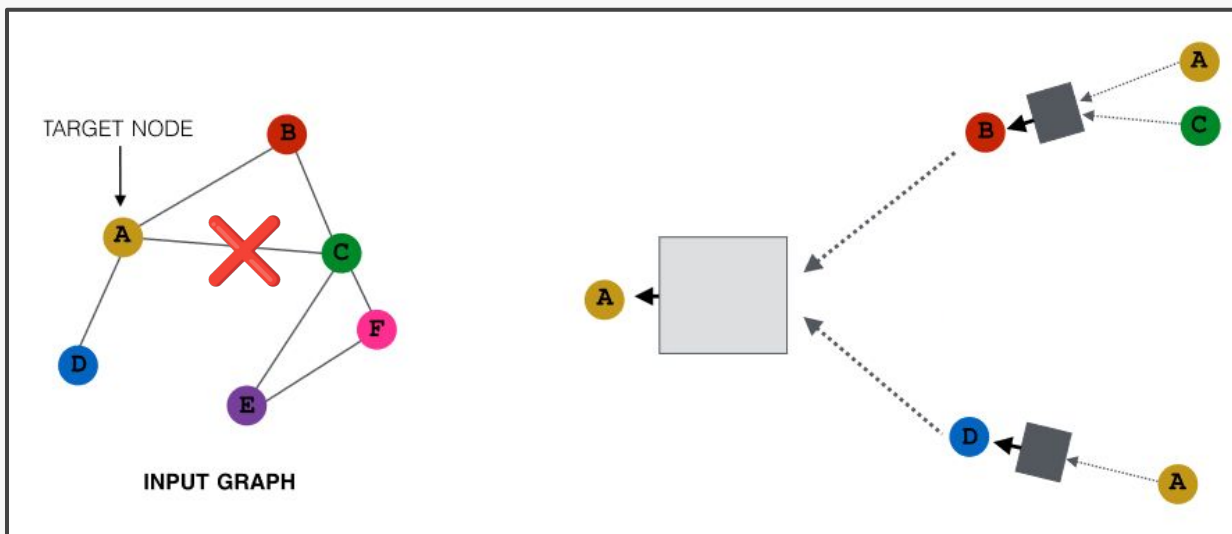
En lugar de usar el grafo original...



Muestreo en vecindarios

Usamos una selección aleatoria de vecinos (para pasar sus mensajes) reduciendo el costo computacional (además, funciona bien en la práctica)

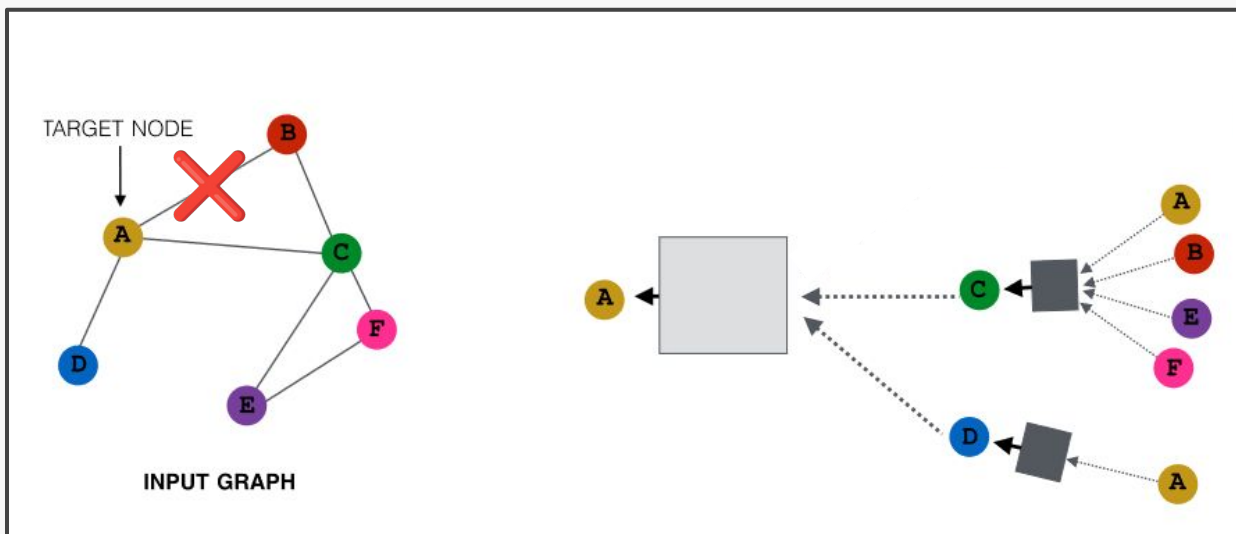
Tomamos solo 2 vecinos de A (B y D)



Muestreo en vecindarios

Usamos una selección aleatoria de vecinos (para pasar sus mensajes) reduciendo el costo computacional (además, funciona bien en la práctica)

La próxima iteración tomamos otro par (C y D)



Agenda

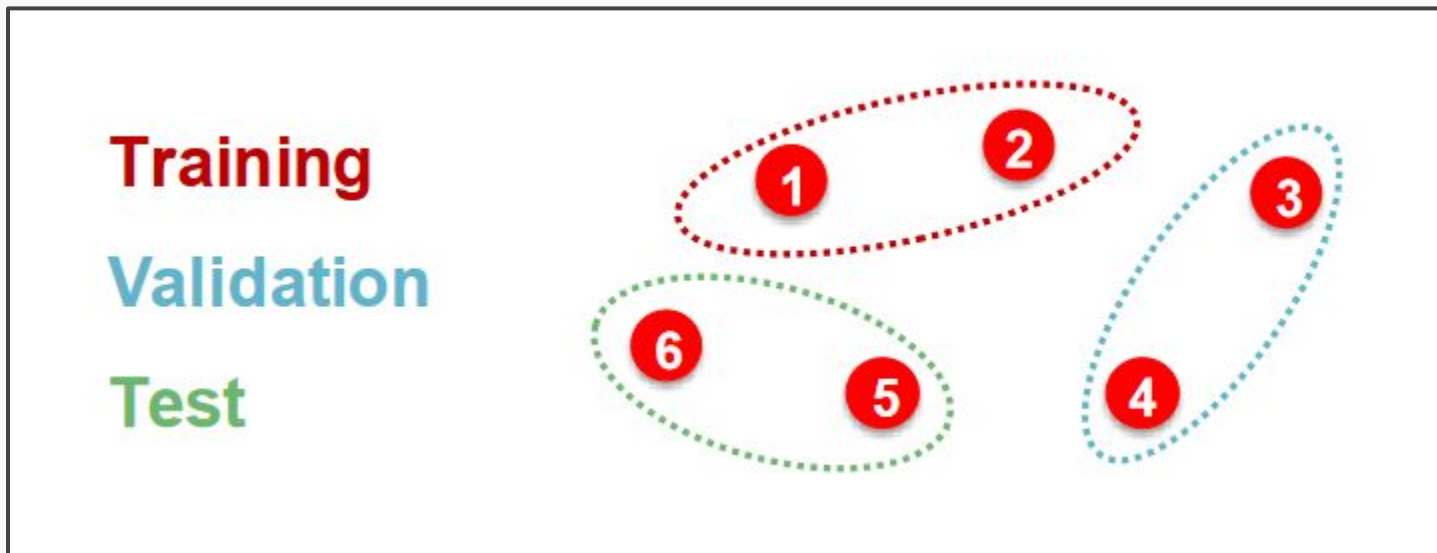
Data augmentation

✨ ***Data splits***

Predicción con GNNs

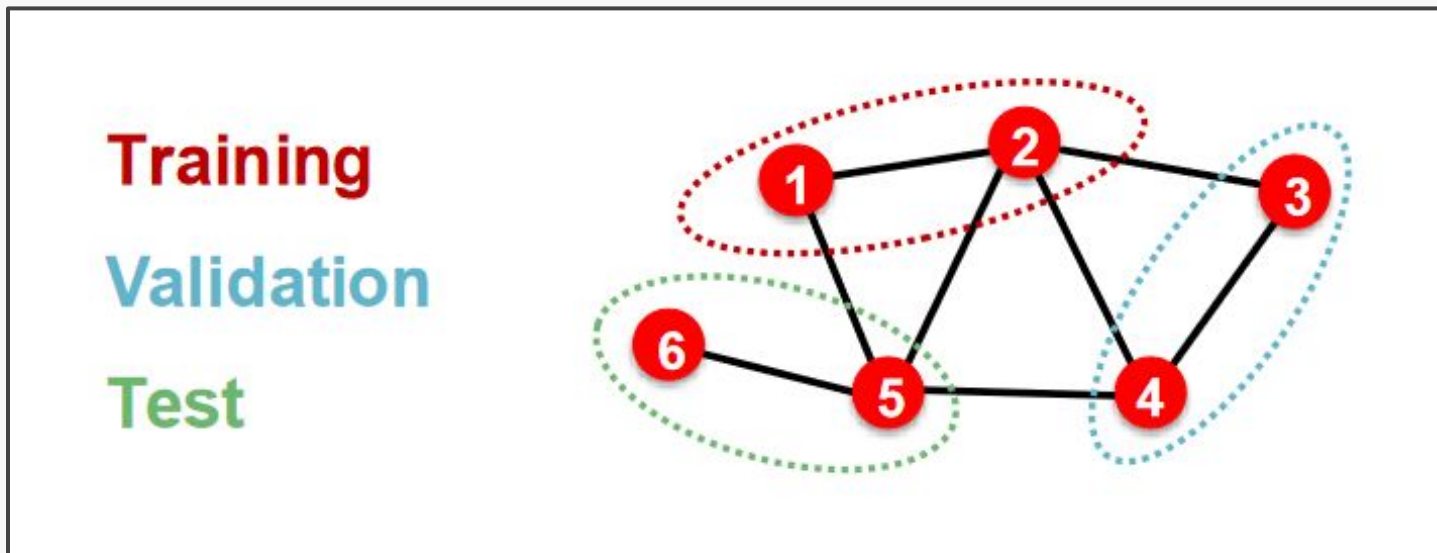
¿Por qué hacer *data splits* en grafos es difícil?

Supongamos que queremos clasificar imágenes: cada punto es una imagen. Los puntos son independientes entre sí: un punto no afectará la predicción de otro



¿Por qué hacer *data splits* en grafos es difícil?

En un grafo, la situación cambia: un nodo puede afectar la predicción de otros nodos, porque va a participar en el paso de mensajes



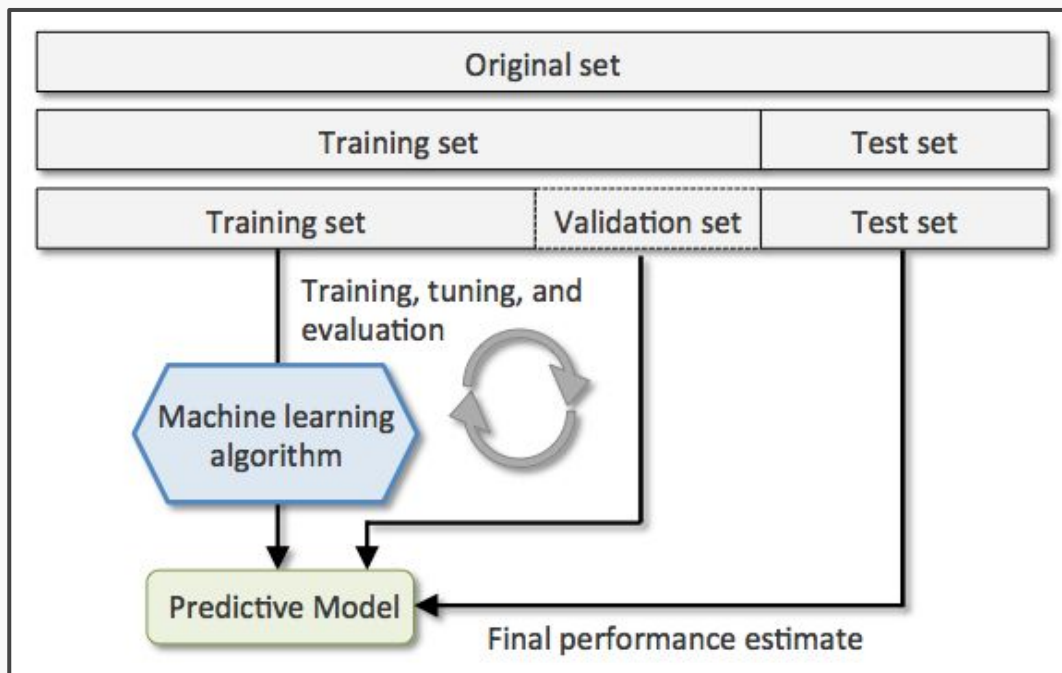
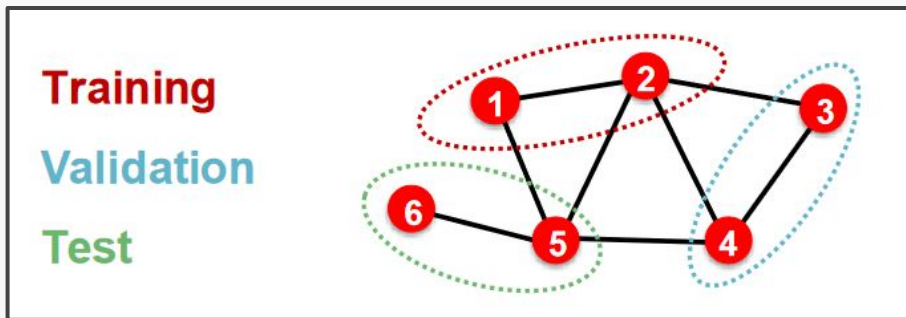


Figura: [Cross Validation Vs Train Validation Test](#) | Cross Validated

Contexto transductivo

En un **dominio transductivo**, el grafo de entrada se observa en todos los *splits* (entrenamiento, validación y test): solo hacemos **split sobre etiquetas**

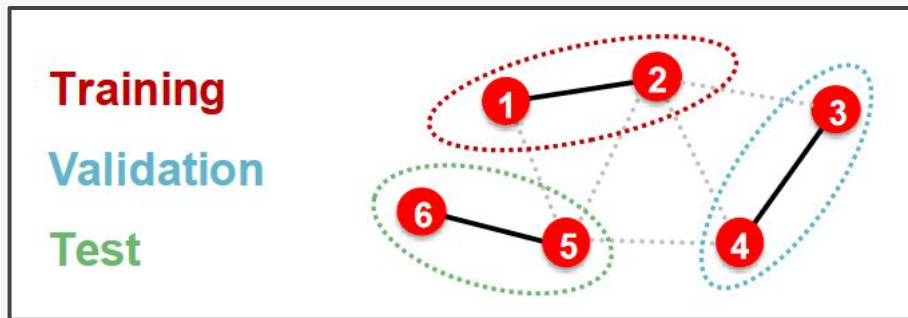
- Al **entrenar**, calculamos los *embeddings* usando todo el grafo pero entrenamos solo con las **etiquetas de los nodos 1 y 2** (figura)
- Al **validar**, calculamos los *embeddings* usando todo el grafo pero evaluamos la performance al predecir la **etiqueta de los nodos 3 y 4** (figura)



Contexto inductivo

En un **dominio inductivo**, el grafo de entrada puede variar y ser diferente en los distintos *splits*: **tenemos 3 grafos que son independientes**

- Al **entrenar**, calculamos los *embeddings* usando el sub-grafo con los nodos 1 y 2, **entrenando con las etiquetas de los nodos 1 y 2** (figura)
- Al **validar**, calculamos los *embeddings* usando el sub-grafo con los nodos 3 y 4, evaluando la **performance sobre los nodos 3 y 4** (figura)



Comparación de contextos

Transductivo: *Data splits* ocurren sobre el mismo grafo

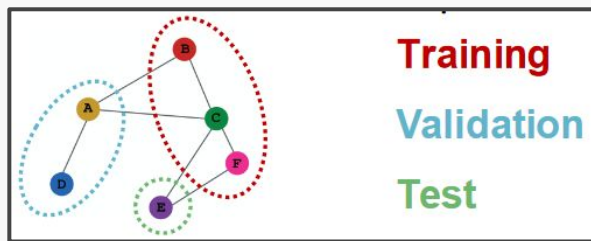
1. Dataset contiene un (1) grafo
2. Todo el grafo es visible en todos los *splits*, ya que solo hacemos *split* de etiquetas
3. Solo es aplicable a tareas de predicción de arcos y nodos (no grafos)

Inductivo: *Data splits* ocurren sobre distintos grafos

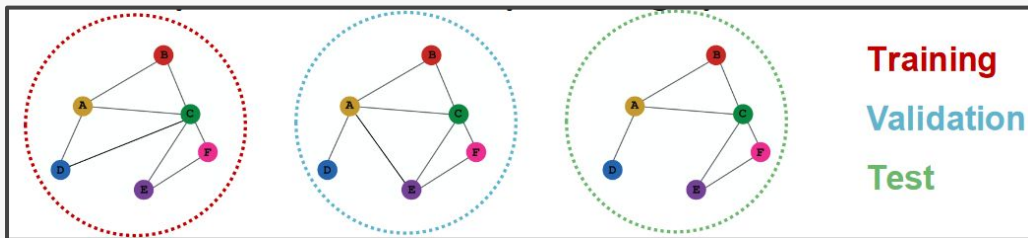
1. El dataset contiene múltiples grafos
2. Cada *split* solo puede observar el grafo de ese *split*. Un modelo deberá generalizar a grafos que no haya visto
3. Aplicable a tareas sobre nodos, arcos y grafos

Clasificación de nodos

Transductivo: Todos los *splits* ven la misma estructura del grafo, pero solo ven las etiquetas de sus nodos respectivos (*split* sobre etiquetas)

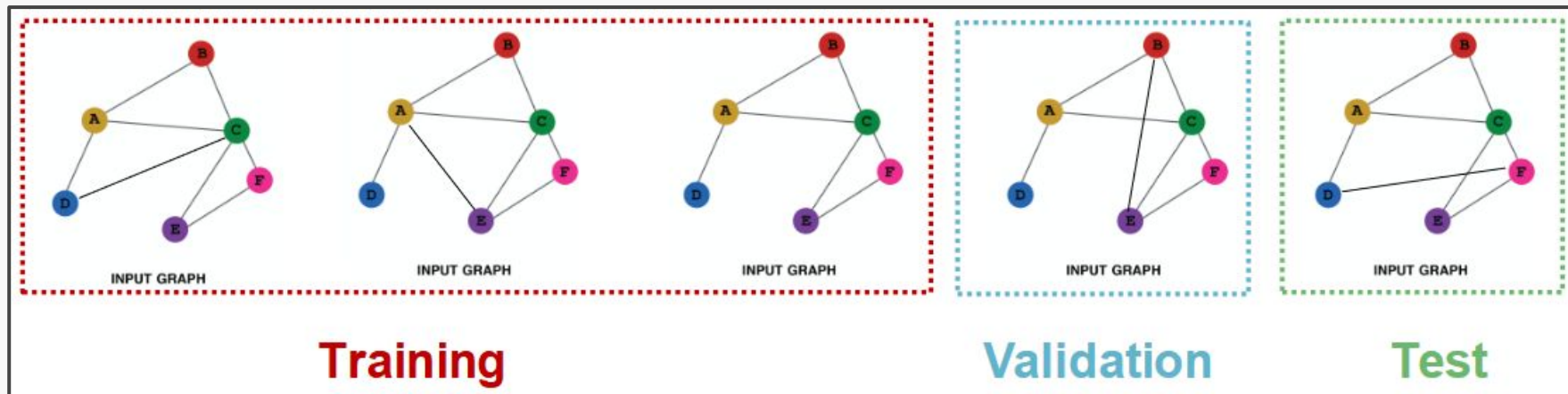


Inductivo: Si tuviéramos un dataset con 3 grafos, cada *split* tendría uno



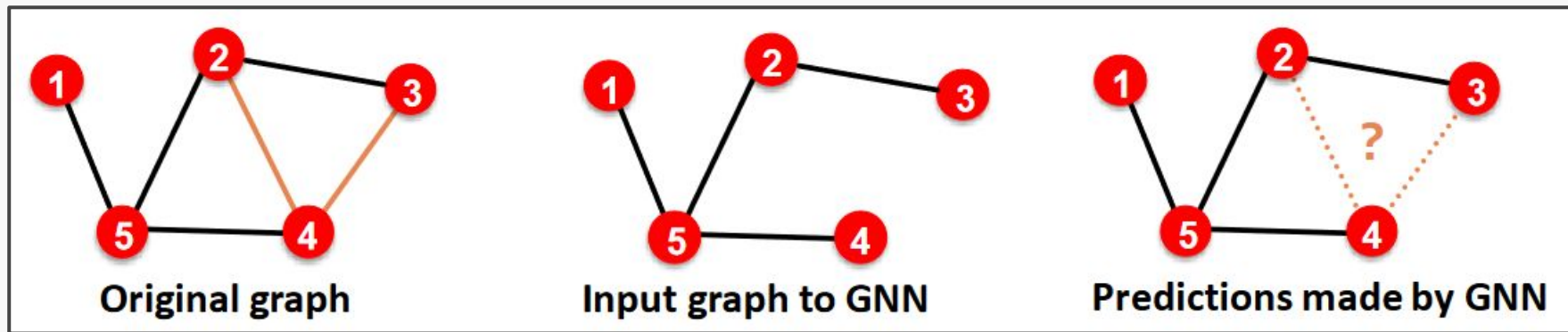
Clasificación de grafos

Solo la configuración **inductiva** está bien definida para clasificación de grafos, porque necesitamos que el conjunto de test sea compuesto de grafos no-vistos



Clasificación de arcos

El objetivo es predecir arcos faltantes, por lo que necesitamos ocultarle arcos a la GNN y permitir que prediga la existencia de esos arcos

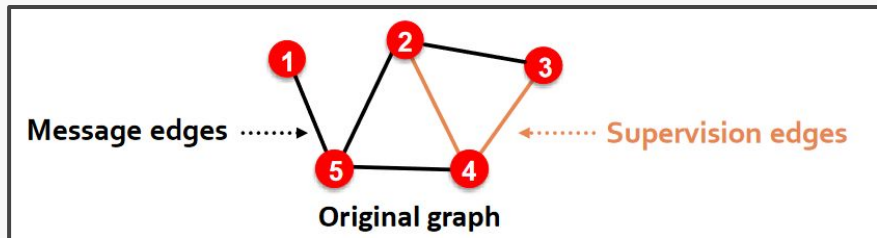


Clasificación de arcos

Paso 1: Definir dos tipos de arcos en el grafo original:

- **Arcos de mensaje:** usados para pasar mensajes en la GNN
- **Arcos de supervisión:** usados como objetivos durante el entrenamiento

Después del paso 1, sólo los arcos de mensaje se mantendrán en el grafo. Los arcos de supervisión deberán ser predichos y evaluados (no pasarán por la GNN)

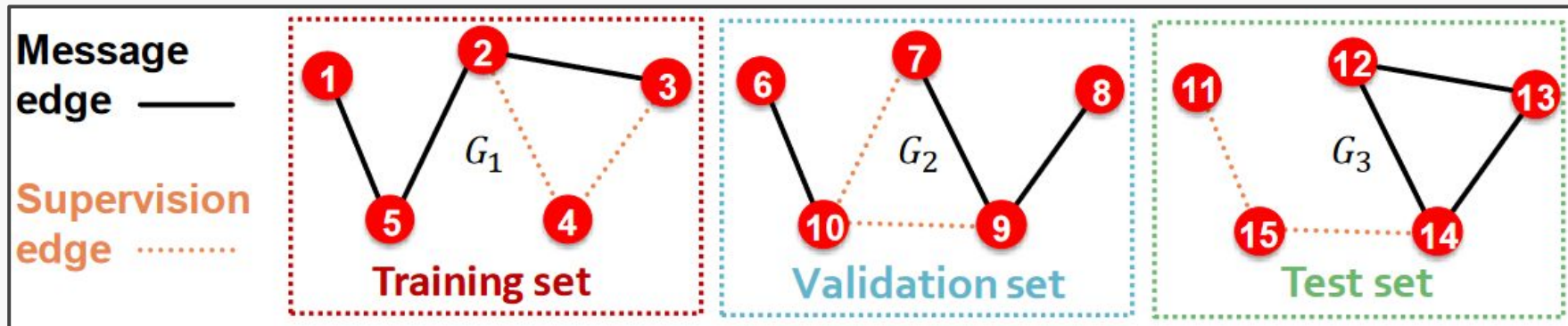


Clasificación de arcos

Paso 2: Se dividen los arcos en conjuntos de cada *split*

Opción 1: *Split* de arcos de tipo **inductivo**: supongamos que tenemos un dataset de 3 grafos, cada *split* tendrá un grafo independiente.

Luego, cada grafo tendrá 2 tipos de arco: de mensajes y de supervisión

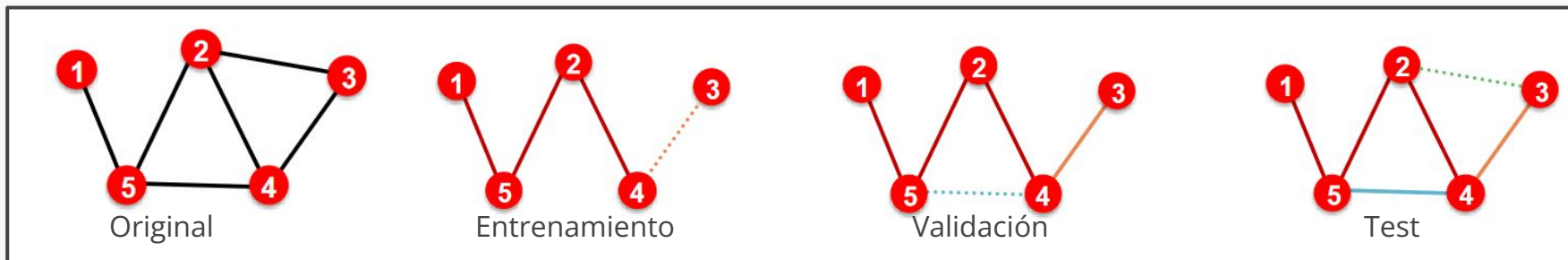


Clasificación de arcos

Paso 2: Se dividen los arcos en conjuntos de cada *split*

Opción 2: *Split* de arcos de tipo **transductivo**:

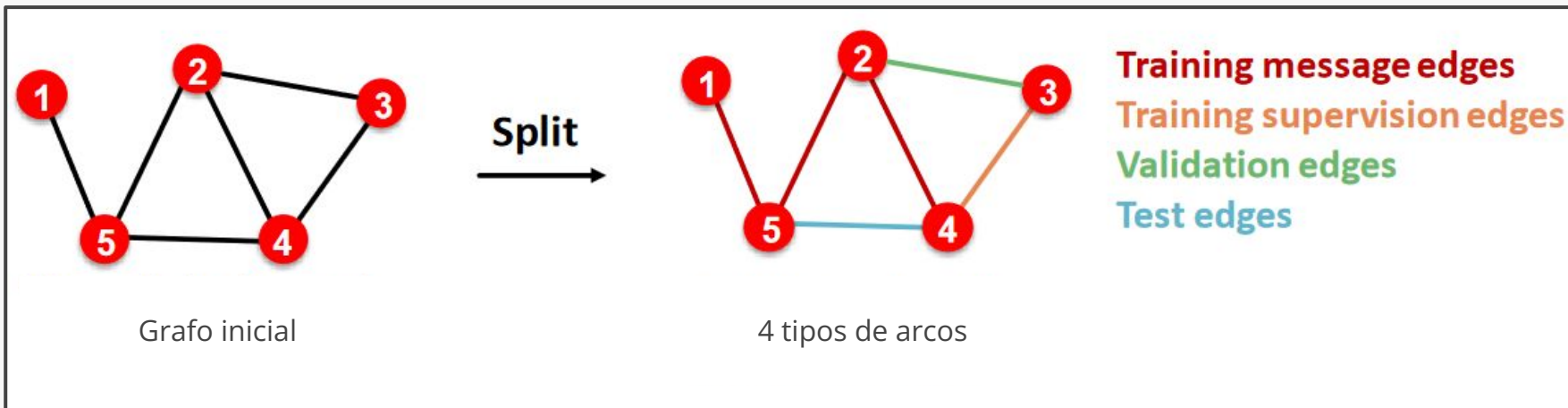
- Al entrenar: usamos los arcos de mensaje de entrenamiento para predecir los arcos de supervisión durante entrenamiento
- Al validar: usamos todos los anteriores para predecir los arcos de validación
- Al hacer test: usamos todo lo anterior para predecir los arcos de test



Clasificación de arcos

Paso 2: Se dividen los arcos en conjuntos de cada *split*

Opción 2: *Split* de arcos de tipo **transductivo**:



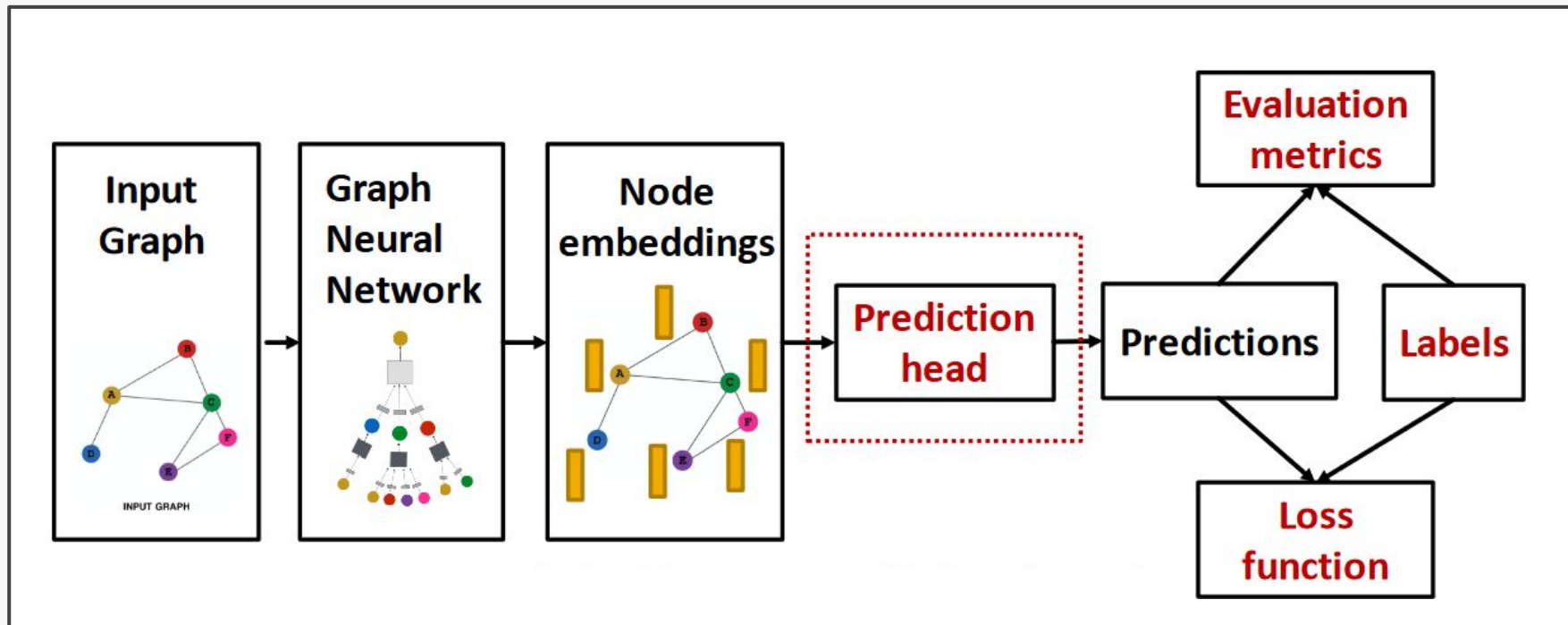
Agenda

Data augmentation

Data splits

✨ **Predicción con GNNs**

Predicción con GNNs



Tipos de tareas

C1

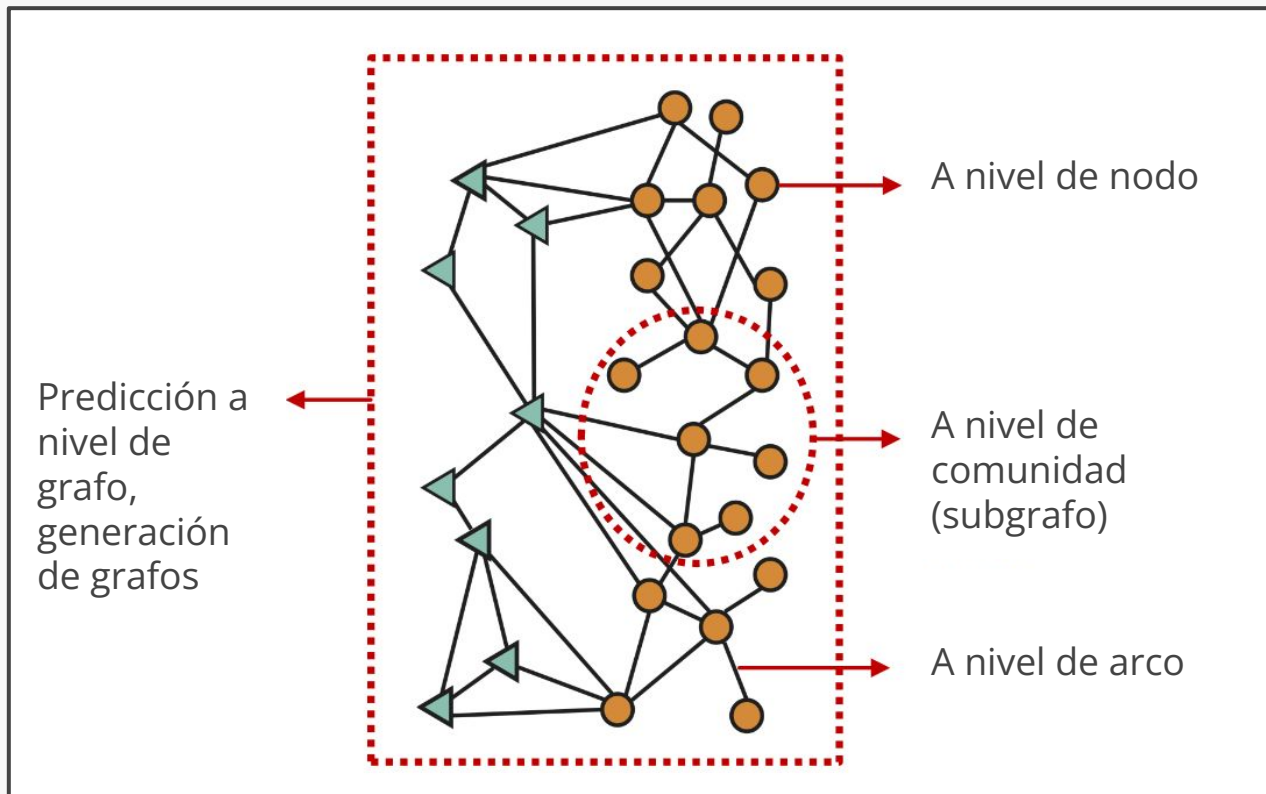


Figura: [CS224W: Machine Learning with Graphs \(Jure Leskovec\) @ Stanford](#)

Predicción a nivel de nodo

Con la GNN, tenemos un *embedding* de nodos:

$$\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\}$$

Si quisiéramos **clasificar o predecir un nodo** usando el embedding:

$$\hat{\mathbf{y}}_v = \text{Head}_{\text{node}}(\mathbf{h}_v^{(L)}) = \mathbf{W}^{(H)} \mathbf{h}_v^{(L)}$$

- $\mathbf{W}^{(H)} \in \mathbb{R}^{k \times d}$: Mapea *embeddings* de nodos desde $\mathbf{h}_v^{(L)} \in \mathbb{R}^d$ a $\hat{\mathbf{y}}_v \in \mathbb{R}^k$ para que podamos calcular la función de pérdida (*loss*)

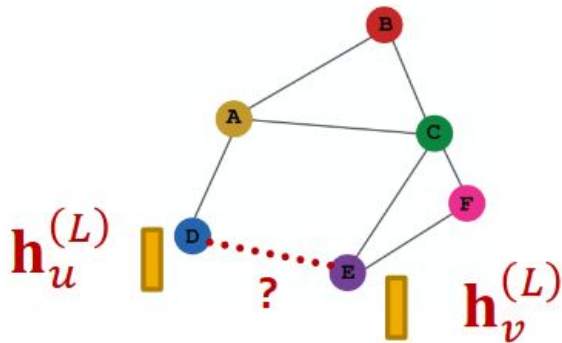
Predicción a nivel de arco

Con la GNN, tenemos un *embedding* de nodos:

$$\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\}$$

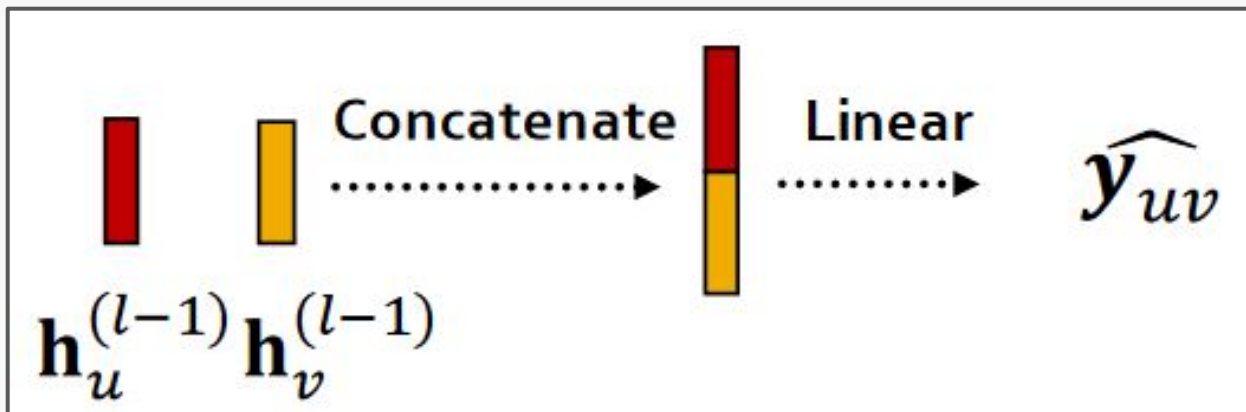
Si quisiéramos **clasificar o predecir un arco** usando el embedding:

$$\hat{y}_{uv} = \text{Head}_{\text{edge}}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)})$$



Predicción a nivel de arco

Opción 1: Concat. + Linear/Dense



Predicción a nivel de arco

Opción 2: Producto punto (~distancia)

$$\hat{y}_{uv} = (\mathbf{h}_u^{(L)})^T \mathbf{h}_v^{(L)}$$

- Solo aplica directamente para predecir existencia de arco (retorna escalar)
- Se puede aplicar a otras tareas, pero requiere una capa adicional:

$$\begin{aligned}\hat{y}_{uv}^{(1)} &= (\mathbf{h}_u^{(L)})^T \mathbf{W}^{(1)} \mathbf{h}_v^{(L)} \\ &\dots \\ \hat{y}_{uv}^{(k)} &= (\mathbf{h}_u^{(L)})^T \mathbf{W}^{(k)} \mathbf{h}_v^{(L)} \\ \hat{\mathbf{y}}_{uv} &= \text{Concat}(\hat{y}_{uv}^{(1)}, \dots, \hat{y}_{uv}^{(k)}) \in \mathbb{R}^k\end{aligned}$$

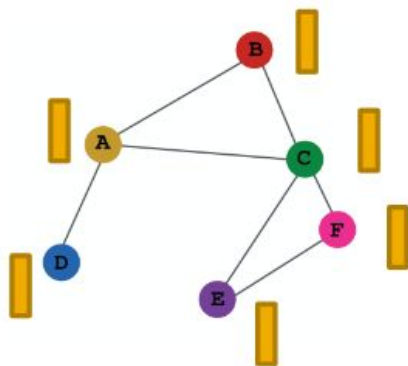
Predicción a nivel de grafo

Con la GNN, tenemos un *embedding* de nodos:

$$\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\}$$

Si quisiéramos clasificar o predecir **un grafo** utilizando el embedding:

$$\hat{\mathbf{y}}_G = \text{Head}_{\text{graph}}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$$



Pred. a nivel de nodo



Predicción a nivel de grafo

- (1) Global mean pooling

$$\hat{\mathbf{y}}_G = \text{Mean}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$$

- (2) Global max pooling

$$\hat{\mathbf{y}}_G = \text{Max}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$$

- (3) Global sum pooling

$$\hat{\mathbf{y}}_G = \text{Sum}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$$

Problemas al usar *pooling*

En grafos grandes, aplicar estas técnicas de *pooling* pierden información:

Por ejemplo, tomemos dos grafos con *embeddings* evidentemente distintos...

$$G_1: \{-1, -2, 0, 1, 2\}$$

$$G_2: \{-10, -20, 0, 10, 20\}$$

$$G_1: \hat{y}_G = \text{Sum}(\{-1, -2, 0, 1, 2\}) = 0$$

¡Iguales!

$$G_2: \hat{y}_G = \text{Sum}(\{-10, -20, 0, 10, 20\}) = 0$$

Pooling jerárquico

Una solución posible es **agregar los *embeddings*** de nodos **jerárquicamente**

1. Agregamos por separado los primeros 2 nodos y los últimos 3 nodos
2. Agregamos estos resultados nuevamente para producir la salida final

$$\{-1, -2, 0, 1, 2\}$$

$$\hat{y}_a = \text{ReLU}(\text{Sum}(\{-1, -2\})) = 0$$

$$\hat{y}_b = \text{ReLU}(\text{Sum}(\{0, 1, 2\})) = 3$$

$$\hat{y}_G = \text{ReLU}(\text{Sum}(\{y_a, y_b\})) = 3$$

$$\{-10, -20, 0, 10, 20\}$$

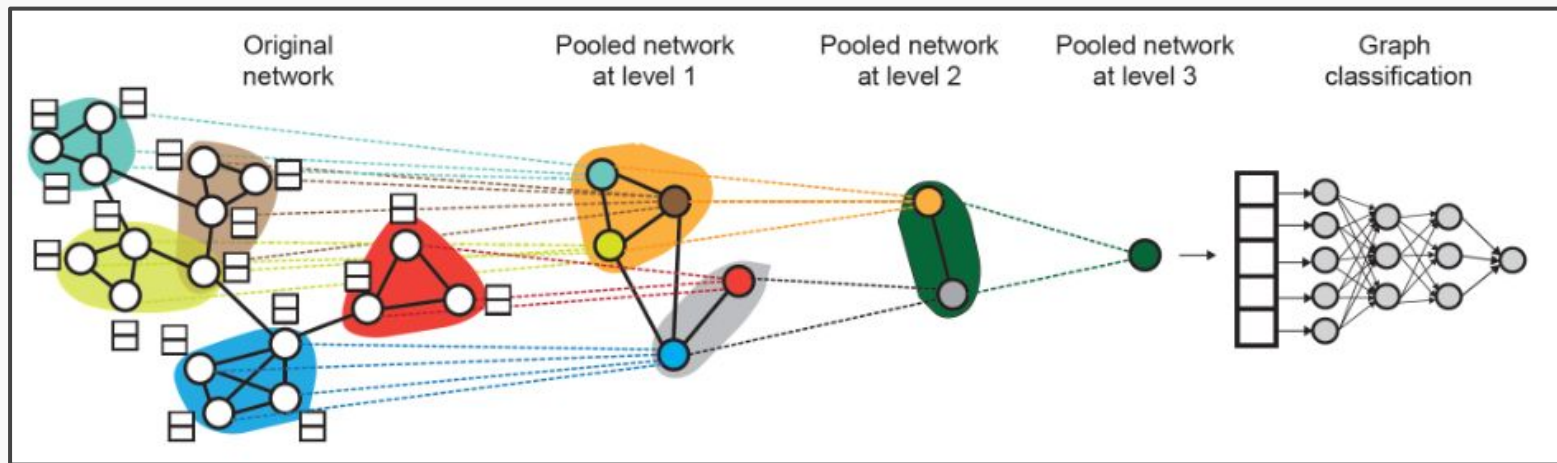
$$\hat{y}_a = \text{ReLU}(\text{Sum}(\{-10, -20\})) = 0$$

$$\hat{y}_b = \text{ReLU}(\text{Sum}(\{0, 10, 20\})) = 30$$

$$\hat{y}_G = \text{ReLU}(\text{Sum}(\{y_a, y_b\})) = 30$$

¡Distintos!

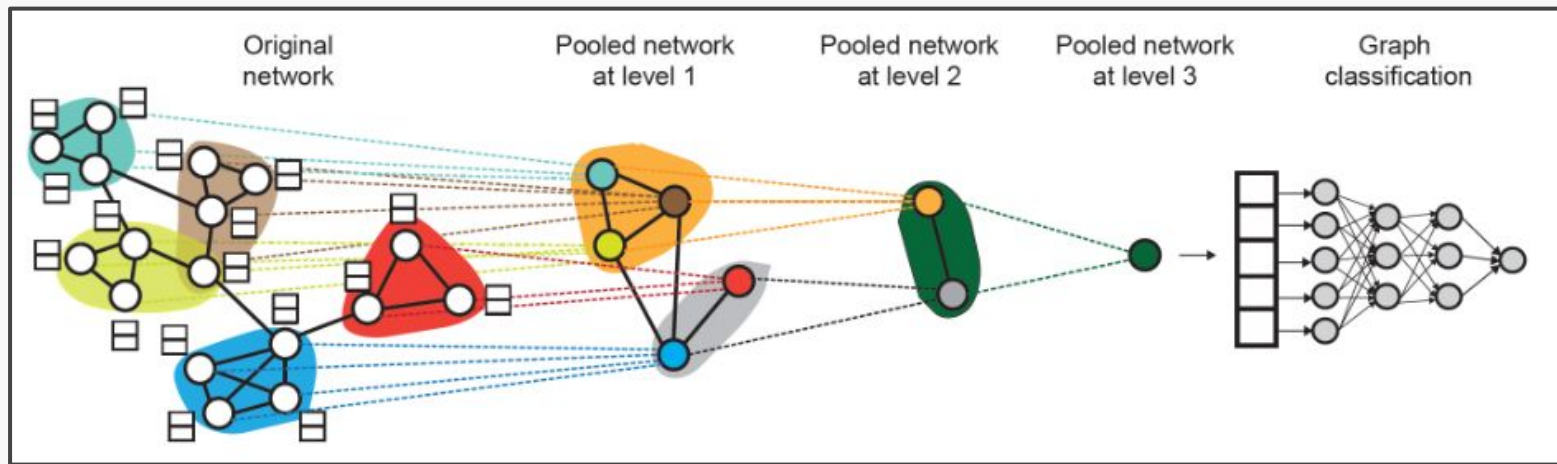
DiffPool para pooling jerárquico



GNN A: Para calcular *embeddings* de nodos

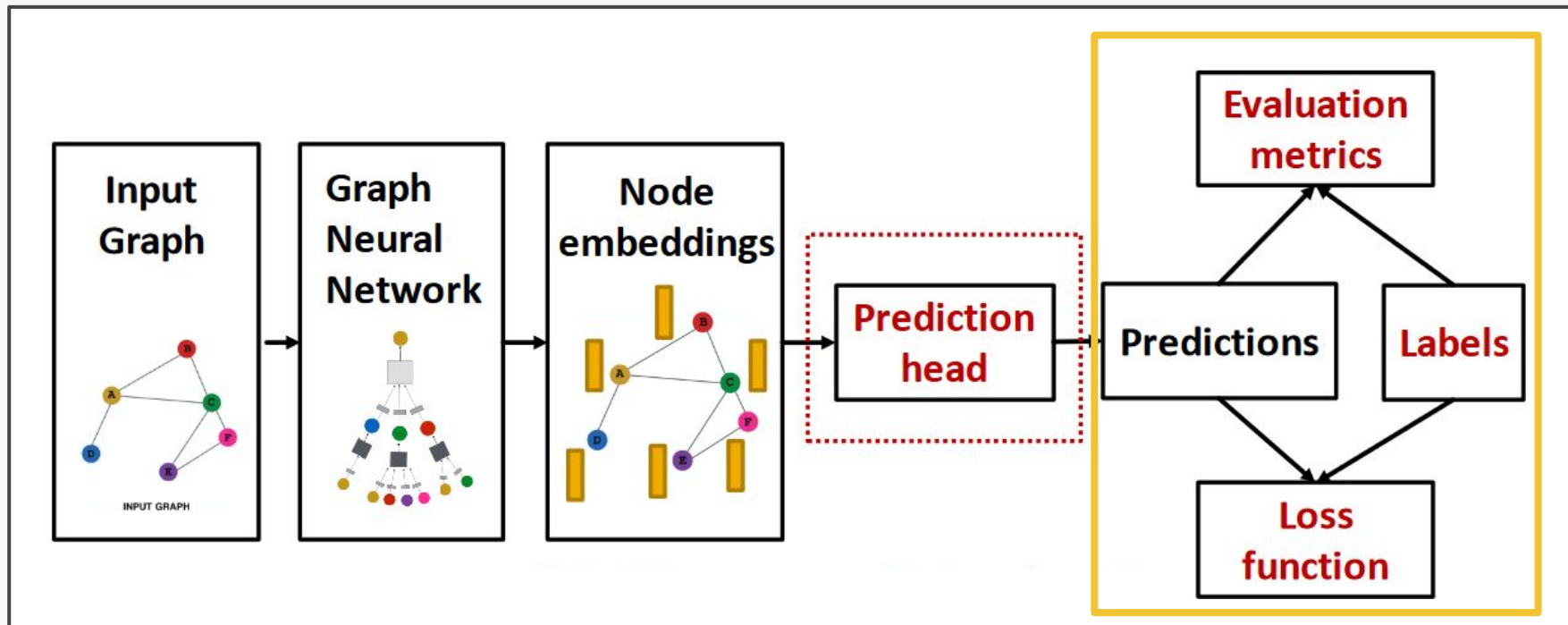
GNN B: Para calcular a qué clúster pertenece cada nodo

DiffPool para pooling jerárquico



En cada pooling, se usan las asignaciones de clúster de GNN B para agregar los embeddings de nodos generados por GNN A. Cada clúster se representa como un nuevo nodo que toma su lugar, y que posteriormente se agrega para generar un nuevo grafo.

Predicción con GNNs



¿Qué les parece la Evaluación Práctica 2?

Dificultad percibida



Tiempo que creo que tomará



Interesante/entretenida



Nota esperada



Poco

Alto

¿Comentarios de la Evaluación Práctica 2?

no ahora, seguramente
al comenzar a hacerla si
:D

las notas de la práctica 1

tiene sentido usar Neo4j
para alguna parte de la
EP2?

¡Gracias por sus respuestas!

¡Éxito con la Evaluación Práctica 2!

Clase 7.1

Entrenando GNNs

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl